Student Attendance Management System Report

1. Title Page:

INSTITUTE OF ENGINEERING AND MANAGEMENT

COURSE NAME: ADVANCED PROGRAMMING (OOPS)

PROJECT TITLE: STUDENT ATTENDANCE MANAGEMENT SYSTEM

STUDENT NAME: SAMRAT PAUL

ROLL NUMBER: 57

INSTRUCTOR NAME: Dr. Swagatam Basu, Dr. Susovan Jana, Dr. Soumadip Biswas

SEMESTER: SPRING 2026

2. Problem Statement:

Maintaining student attendance manually in registers is time-consuming and can lead to errors or loss or loss of records. Teachers often face difficulty in tracking attendance for many students and attendance percentages accurately. The Student Attendance Management System is designed to provide This project implements a Library Management System using Java with a graphical user interface. It a simple digital solution for recording and managing student attendance. The system allows teachers to add student details, mark daily attendance, and view attendance records easily. By using a Java-based application with Object-Oriented Programming concepts, the system helps improve accuracy, organization, and efficiency in managing attendance data.

3. System Design:

3.1 UML Class Diagram:

3.2 Description of Major Classes:

- **Person Class:** Stores basic details like id and name for reuse by other classes
- **Student Class:** Extends Person and manages attendance records and percentage calculation
- **AttendanceService Class:** Handles business logic such as adding students and marking attendance.
- **MainFrame Class:** Provides GUI interface for user interaction and displays output interaction.

3.2 Package Structure:

- model → Person and Studennt class
- service → AttendanceService class
- gui → Mainframe class
- main → Main class

3.3 Interaction Flow:

1. User enters student ID and name to add a new student.
2. System stores the student details in the list.
3. User selects student ID and marks attendance as Present or Absent.
4. System updates attendance records for the selected student.
5. User requests to view attendance details
6. System calculates and displays attendance percentage.

4. OOP Concepts Demonstrated:

4.1 Encapsulation:

Where: Student class

Why: To protect data from direct access

Snippet:

```
private int id;
private String name;
public int getId()
{ return id; }
public String getName()
{ return name; }
```

Explanation: Data is hidden using private variables and accessed through methods.

4.2 Abstraction:

Where: AttendanceService class

Why: To hide internal logic

Snippet:

```
public void addStudent(Student s) {
students.add(s);
}
```

Explanation: User interacts with methods without knowing internal implementation.

4.3 Inheritance:

Where: Person → Student

Why: To reuse common properties

Snippet:

```
class Person { int id; String name; }
class Student extends Person
{
List<Boolean> attendance;
}
```

Explanation: Student inherits properties from Person class.

4.4 Polymorphism:

Where: Student class

Why: To allow flexible method behavior

Snippet:

```
public double getAttendancePercentage() {
return (present * 100.0) / total;
}
```

Explanation: Method behavior can be modified in future subclasses.

5. Implementation Highlights:

5.1. Java is used as the core programming language to develop the system.

5.2. The application follows a modular structure using model, service, gui, and main packages.

5.3. Object-Oriented Programming concepts are applied for better design and maintainability.

5.**4.** ArrayList is used to store and manage student records dynamically.

5.5. Java Swing is used to create a simple graphical user interface.

5.6. Exception handling is implemented to manage invalid inputs and

5.7. Attendance percentage is calculated automatically based on stored records.

5.3 GUI Interaction:

• Designed simple user interface

• Added input fields for ID and name

• Used buttons for operations

• Displayed results in text area

6. Testing & Error Handling:

- Test Cases:

  • Initialized git repository
  • Created project structure
  • Committed code regularly
  • Fixed bugs and updated code

- Edge Cases:

I. No student in system
II. Empty input fields

- Failure Scenarios

I. Invalid student ID
II. No attendance

7. Git Workflow:

• Initialized git repository
• Created project structure
• Committed code regularly
• Fixed bugs and updated code

8. Conclusion & Future Scope:

• Developed a simple attendance management system
• Reduced manual work and improved accuracy
• Applied OOP concepts for better design
• Provided user-friendly interface for easy use

Future Scope:

- Add database for permanent
- Improve GUI with advanced design
- Implement login system for users
- Convert application into web or mobile

Appendix:

1. Copy of the initial project proposal

Project Title: Student Attendance Management System using Java

Problem Statement:

Maintaining student attendance manually in registers is time-consuming and can lead to errors or loss of records. Teachers often face difficulty in tracking attendance for many students and calculating attendance percentages accurately. The Student Attendance Management System is designed to provide a simple digital solution for recording and managing student attendance. The system allows teachers to add student details, mark daily attendance, and view attendance records easily. By using a Java-based application with Object-Oriented Programming concepts, the system helps improve accuracy, organization, and efficiency in managing attendance data.

Target User:

- Teachers
- School administrators
- College faculty members
- Educational institutions

Core Features:

- Add student details
- Mark daily attendance (Present/Absent)
- View student attendance records
- Calculate attendance percentage
- Display list of students and their attendance status
- Simple menu-driven interface

OOP Concepts to be Used:

- **Abstraction:** The system will abstract attendance operations through service classes that handle attendance recording and student management without exposing internal data handling.
- **Inheritance:** A base **Person or Student class** can be extended to represent different roles such as regular students or special category students.
- **Polymorphism:** Methods such as display or attendance calculation can behave differently depending on the class implementation.
- **Exception Handling:** Try–catch blocks will handle invalid inputs, missing student records, or incorrect attendance entries to prevent program crashes.
- **Collections / Threads:** Java **ArrayList** collection will be used to store and manage multiple student records dynamically.

Proposed Architecture Description:

The system will follow a **modular architecture** where different components of the application are separated into logical packages such as **model, service, and main**. The **model layer** represents the student data and attendance information, the **service layer** manages operations such as adding students and marking attendance, and the **main layer** handles the user interface and interaction with the system. This modular structure improves maintainability, readability, and scalability of the program while clearly separating responsibilities across different components.

2. Copy of the review mail received (with APPROVED_MINOR)

S

soumadip.biswas@ieminterna... 10 Mar

Samrat Paul

Dear Samrat Paul,

Project: Student Attendance Management System using java
Decision: MAJOR_REVISION
Score: 70

Strengths:
The proposal clearly defines a practical and relevant problem that a Student Attendance Management System can solve. The target audience is well-identified. The proposed features are appropriate for a foundational Java OOP project and provide a clear scope. The modular architecture (model-service-main) is a strong point, promoting good design principles like separation of concerns, maintainability, and scalability. The inclusion of core OOP concepts such as Abstraction, Inheritance, Polymorphism, Exception Handling, and Collections demonstrates a good understanding of Java OOP fundamentals.